

VIT®
CHENNAI
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

VELLORE INSTITUTE OF TECHNOLOGY
CHENNAI

REVIEW-2 PROJECT REPORT

NutriChef

An NLP-Based Intelligent Recipe Discovery
and Recommendation System

Natural Language Processing
Course Code: ISWE310L

Submitted by
S. Divyabanu
Registration Number: **23MIS1002**

Academic Year 2026-2027

Project Repository
https://github.com/sDivyabanu/NutriChef

Abstract and Contents

NutriChef is a locally executed recipe discovery platform that converts everyday English and selected Tamil-English expressions into structured cooking requests. Conventional keyword search does not reliably distinguish ingredients a user owns from ingredients that must be included or excluded. NutriChef addresses this problem through a curated ingredient lexicon, quantity-preserving preprocessing, rule-based intent and constraint parsing, RapidFuzz spelling correction, TF-IDF retrieval and pretrained sentence embeddings.

The inspected corpus contains 13,475 imported recipes from 13,501 source rows. A hybrid ranker applies text-derived constraints before combining lexical, semantic and ingredient-coverage signals. The application also implements structured conversational state, recipe adaptation, local-model generation, content-based recommendations and heuristic meal planning. The pretrained embedding model is used for inference; no project-specific neural training was found.

Saved held-out results report ingredient name-and-role F1 of 0.905 on 35 queries, intent accuracy of 0.967 on 30 queries and hybrid reported nDCG@10 of 0.819 on 15 retrieval queries. These are historical results on small, AI-authored evaluation sets, not independently human-validated accuracy claims. The evaluator uses a fixed ideal-ten denominator for nDCG. A live planning check also exposed a dietary inconsistency, discussed explicitly in this report.

Keywords: ingredient normalisation, TF-IDF, MiniLM, hybrid retrieval, conversational recommendation.

Contents

Section	Page
1. Introduction; 2. Problem Statement	3
3. Literature Review	4
4. Dataset Description	5
5. Proposed System and Architecture	6
6. Methodology: query processing and normalisation	7
6. Methodology: lexical and semantic retrieval	8
6. Methodology: hybrid ranking	9
6. Methodology: assistant and recommendations	10
6. Methodology: modification and generation	11
6. Methodology: meal planning and shopping list	12
7. Implementation and application evidence	13-14
8. Experimental Results	15-16
9. Discussion; 10. Conclusion; 11. Future Scope	17
12. References and repository evidence	18

1. Introduction

Recipe discovery is an information-retrieval task with practical constraints. A cook may describe a pantry, a dietary preference, a time budget and an exclusion in a single sentence. The system must preserve these relationships instead of treating every word as a positive search term. For example, a request containing "without peanuts" should not become a recommendation for peanut-based dishes merely because the ingredient is semantically relevant.

NutriChef provides a natural-language interface over a local recipe collection. The frontend presents recipes, parsed filters, ingredient coverage, explanations and modification controls. The FastAPI service performs domain-specific text processing and retrieval. PostgreSQL supports application account data, while a separate SQLite store supports recipes and NLP-related user records. This division is central to the actual architecture [9].

1.1 Project objectives

The implemented objectives are to interpret ingredient roles and cooking constraints; normalise synonyms and selected transliterated Tamil expressions; tolerate common domain spelling errors; retrieve and rank recipes using complementary lexical and dense representations; preserve constraints across conversational turns; and expose recipe adaptation, recommendation and planning through a usable interface.

The project uses a mixture of deterministic NLP, classical statistical text representation and pretrained neural inference. It should therefore be described as a hybrid NLP application rather than a newly trained large language model. The conversational assistant itself is rule-based and database-grounded.

2. Problem Statement

Traditional substring matching is brittle when an ingredient appears under a different name, a spelling variant or a regional alias. It also lacks an explicit representation of negation, dietary intent and unavailable ingredients. A query such as "I have tomatoes and rice, no onion" contains both positive and negative evidence that must be separated before ranking.

The engineering problem is to transform such requests into auditable structured constraints, retrieve useful candidates from incomplete recipe data and explain why a result is recommended. This must be done without inventing nutritional values, equating semantic similarity with dietary suitability or overstating support for multilingual conversation.

2.1 Scope of this review

This report describes the local working tree inspected for Review-2, the saved evaluation artefacts and selected live application checks. It does not certify every user workflow, train any model or rerun the full retrieval or generative benchmark. The hosted GitHub repository may differ from the uncommitted local implementation. No application source, dataset or benchmark logic was changed while preparing this report.

3. Literature Review

3.1 Traditional information retrieval and TF-IDF

The vector-space approach represents a query and a document with term weights and compares their directions. TF-IDF gives greater importance to terms that distinguish documents within a collection; cosine similarity reduces dependence on document length [1]. This approach is efficient and interpretable, but exact vocabulary overlap remains important. NutriChef supplements it with ingredient aliases, lightweight stemming and dense retrieval.

3.2 Sentence embeddings and semantic retrieval

Sentence-BERT derives sentence representations that can be compared by cosine similarity, allowing document representations to be computed in advance [2]. The all-MiniLM-L6-v2 model card describes a 384-dimensional representation suitable for semantic search [3]. NutriChef uses this pretrained model locally. Semantic resemblance alone does not establish that a result satisfies an exclusion or dietary constraint.

3.3 Fuzzy matching for domain spelling

Damerau-Levenshtein distance measures edit operations, including adjacent transposition. RapidFuzz provides efficient distance and candidate-search functions [4]. NutriChef uses bounded edit distance over domain terms before consulting frequent corpus words. This is a constrained correction mechanism, not a general language model or an unrestricted replacement of every unfamiliar token.

3.4 Hybrid retrieval

Bruch, Gai and Ingber analyse fusion of lexical and semantic retrieval scores and show why the fusion function and its parameters deserve evaluation [5]. NutriChef follows the broader principle of combining complementary evidence, but its implemented ranker is a hand-weighted scoring system with additional ingredient and preference features. It is not a reproduction of that paper or an implementation of reciprocal-rank fusion.

3.5 Conversational recommendation

Conversational recommender systems support interactive preference elicitation, questions and refinement instead of a single one-way recommendation list [6]. NutriChef implements this interaction with explicit state, deterministic merge rules, ordinal references and templated replies grounded in stored recipes. The presence of a chat interface does not imply that a generative conversational model is used.

3.6 Position of the present system

The contribution is application-level integration of domain normalisation, structured constraints and explainable retrieval. The small local evaluation supports comparison within this implementation, but it does not establish superiority over published research systems. No external benchmark leaderboard, human preference study or neural fine-tuning result is claimed.

4. Dataset Description

The source file is **data/13k-recipes.csv**. The existing project documentation attributes it to the Epicurious-derived Kaggle collection "Food Ingredients and Recipes Dataset with Images" [7]. The local copy contains an unnamed index column and five named columns. An image-name field is a reference string; actual recipe photographs are not bundled with this inspected corpus.

Attribute	Verified description
Source records	13,501 CSV rows
Imported recipes	13,475 after import filtering
Excluded records	12 without usable ingredients; 14 exact duplicates
Remaining quality issues	2 imported rows have no instructions; 158 titles occur in multiple recipes
Source columns	Unnamed index, Title, Ingredients, Instructions, Image_Name, Cleaned_Ingredients
Canonical ingredient resource	418 lexicon entries; 4,545 normalised surface forms, counted from the current code
Absent source metadata	Nutrition, servings, declared cooking time, cuisine, meal type and diet labels; image files are absent

4.1 Import and cleaning

The importer safely reads the Ingredients field as a Python literal list, trims ingredient strings, rejects rows without ingredients and supplies a missing title from the image slug when necessary. Exact duplicate detection hashes the normalised title, ingredient list and trimmed instructions. Re-import targets source recipes and preserves user-created and generated records. The CSV itself remains unchanged.

A separate recount reproduced the import totals. The CSV SHA-256 also matched **backend/data/corpus_report.json**. The importer reads the original Ingredients field; the existence of Cleaned_Ingredients does not mean that column is used as the NLP ground truth.

4.2 Derived metadata and limitations

Ingredient parsing produces canonical items, quantities, optionality, equipment lines and unknown lines. Rules infer diet flags, allergens, cuisine, meal type, difficulty and spice level. Estimated cooking time sums durations stated in instructions and may omit preparation. These values are derived estimates, not supplied annotations or reliable nutritional measurements.

Numeric nutritional constraints can be checked when a recipe has author-entered values. Unknown nutrition is retained and labelled as not verified rather than treated as evidence that the limit is satisfied. The source attribution is documented, but an independent provenance and licence audit was not performed for this report.

5. Proposed System and Architecture

The implementation is a local web application with two persistence systems. Next.js serves the interface and application APIs, while FastAPI owns the recipe-oriented NLP services. The browser calls the same-origin **/backend/*** proxy, which forwards requests to FastAPI under **/api/*** and carries the login cookie.

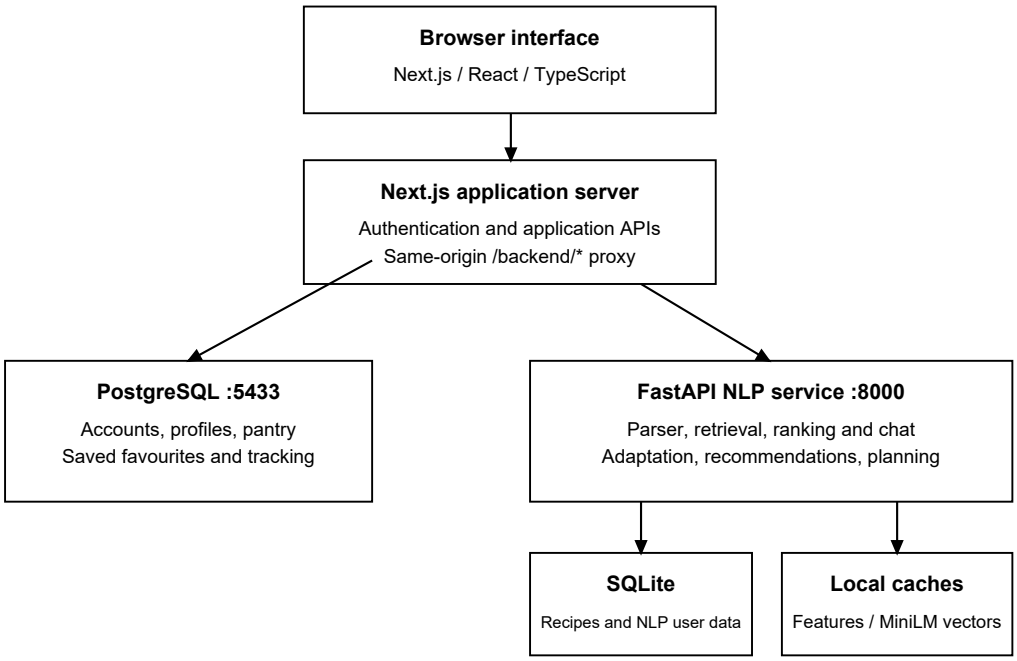


Figure 1. Architecture reconstructed from next.config.ts, Prisma models, backend/app/db.py, services.py and the FastAPI routers. Optional generation and vision services are omitted from the main path for clarity.

5.1 Persistence boundaries

PostgreSQL stores authentication users, profiles, pantry items, saved favourites and hydration/weight records through the application layer. The SQLite recipe store contains source and user recipes, version history, preferences, interactions, search feedback and saved searches. The Prisma schema also contains a Recipe model, but the inspected NLP corpus is served from SQLite; these should not be conflated.

5.2 Runtime lifecycle

A process-wide service container builds the NLP engine in a background thread and reuses it. Feature records are cached with a feature version and text hash. TF-IDF is fitted at engine startup; recipe embeddings are loaded from a model-and-corpus-specific NPZ cache. Create, edit and soft-delete events update the active retrieval index incrementally.

5.3 Optional services

The local llama.cpp server is started lazily for generative requests, unless eager startup is configured. Optional vision detection uses YOLOv8n/COCO and supports only the ingredient-food classes recognised by that configuration. Neither optional component is needed for deterministic parsing or the chat assistant. Runtime model loading is configured for local files.

6. Methodology

6.1 Natural-language query processing

Shared preprocessing lowercases text, folds Latin accents, normalises apostrophes and dashes, converts Unicode fractions to ASCII forms and retains token spans. Quantities are preserved for parsing. The query parser applies spelling correction, extracts regular-expression constraints and phrase dictionaries, identifies ingredient mentions, assigns roles, detects intent and checks conflicts or ambiguity.

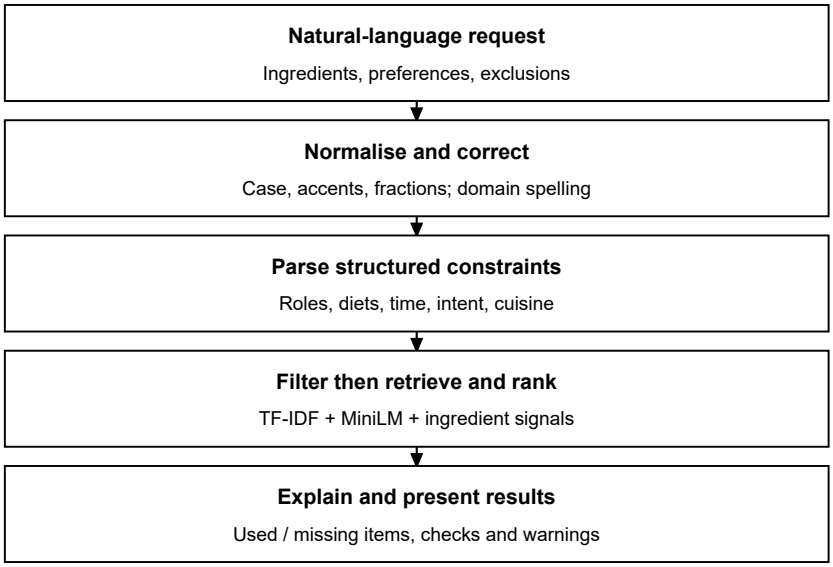


Figure 2. End-to-end query-to-result pipeline. Recipe-side feature extraction and vector construction occur at index time; query-side processing occurs for each request.

6.2 Ingredient normalisation and Tanglish support

The curated lexicon maps canonical names and aliases using greedy longest matching over phrases of up to six tokens. Explicit aliases take precedence over automatically generated plurals. A parent hierarchy distinguishes specific ingredients from broad groups and supplies inherited dietary and allergen facts. Examples in the code include capsicum to bell pepper and kathirikai to eggplant.

Ingredient roles are **available, required, preferred, excluded** and **unavailable**. Cue phrases and negation scope determine their interpretation. Tamil-English support uses a finite alias vocabulary and markers such as vechu for availability and illama for exclusion. It is not unrestricted Tamil translation or general bilingual language understanding.

6.3 Spelling correction

Known lexicon, constraint, Tanglish and sufficiently frequent corpus words are protected. Unknown words of up to three characters are not corrected; lengths four to five permit one edit, and longer words permit two. RapidFuzz searches domain terms first, then frequent general terms, with ties resolved by edit count, initial-letter agreement and corpus frequency.

6.4 TF-IDF and Semantic Retrieval

6.4.1 TF-IDF document construction

The lexical document repeats the recipe title three times and canonical ingredients twice, then includes raw ingredient text and the first 1,500 instruction characters. This intentionally strengthens title and ingredient evidence. A custom analyser normalises text, drops stopwords, applies a small plural/suffix normaliser and emits word unigrams and adjacent bigrams.

TfidfVectorizer uses sublinear term frequency, min_df = 2, max_df = 0.6 and float32 values. With its default smoothed IDF and L2 normalisation [8], the matrix-query dot product is cosine similarity. No supervised relevance labels are used to fit this representation.

Element	Implemented form
Term frequency	tf* = 1 + ln(tf), for a term with positive frequency
Smoothed IDF	idf(t) = ln((1 + N) / (1 + df(t))) + 1
Vector normalisation	Each nonzero weighted vector is scaled to unit L2 length
Lexical similarity	L(q, r) = v(q) dot v(r)
Incremental changes	New or edited recipes use the existing vectoriser vocabulary; a startup refit refreshes it

6.4.2 Pretrained sentence embeddings

The configured default model is **sentence-transformers/all-MiniLM-L6-v2**. Its pretrained encoder produces 384-dimensional sentence vectors [3]. NutriChef encodes the title and up to 25 canonical ingredient names, falling back to up to 15 raw ingredient lines when canonical names are unavailable. Embeddings are L2-normalised, stored as float32 arrays and compared by dot product.

The embedding cache filename includes a sanitised model name and a digest of recipe IDs and embedding-document text. The existing MiniLM cache is approximately 20.8 MB. The model is loaded with local_files_only=True; an unavailable local model triggers a lexical-only fallback rather than a runtime download.

6.4.3 Correct interpretation of learning

The TF-IDF vocabulary and IDF are fitted to the corpus. MiniLM is an externally pretrained model used for inference; no training-from-scratch or project-specific fine-tuning code was found. Additional model candidates are configurable, but no saved comparative results for those alternatives were located. The model card also notes truncation of long inputs, which limits what a compact dense representation can retain.

6.5 Hybrid Retrieval and Ranking

Hybrid search first intersects an active-recipe mask with constraints for recognised exclusions, diet/allergy groups, required ingredients, meal type, cuisine evidence and stated time limits. Profile preferences can add constraints. A pantry query also requires the recipe to use at least one available ingredient. Nutrition values that are missing remain unchecked and are explained as unknown.

6.5.1 Candidate selection

Lexical and semantic arrays are normalised separately by their maximum among eligible candidates. A preliminary score combines these signals and any similar-to signal. Pantry mode additionally rewards the fraction of available items used and favours smaller essential-ingredient sets. Detailed scoring is capped at 600 candidates when there is a text signal; the no-text threshold is 6,000, although the truncation branch itself selects 600. This is a bounded heuristic search, not an exhaustive learned reranker.

Signal	Default weight / implementation
Semantic; lexical; similar-to	1.0; 0.7; 1.0
Available ingredients used	1.4 x used available items / available items
Recipe ingredient coverage	0.8 x covered essential items / essential items
Missing ingredients	-0.5 x min(missing count, 10) / 10
Preferred; unavailable items	+0.5 x preferred fraction; -0.8 per needed unavailable item
Cuisine; meal; quality	0.5; 0.25; 0.3, applied to feature-specific rules
Personalisation	Configured personal weight 0.25; preference-specific scoring
Parse-quality prior	-0.05 x min(unrecognised ingredient lines, 3)

6.5.2 Coverage, explanations and ordering

Essential ingredients exclude optional items and recognised staples, including user staples. Pantry match is the rounded share of essential ingredients covered by what the user has. It is not a semantic similarity score. The final score is the sum of individual contributions; ties use missing-item count and recipe ID. Duplicate normalised titles are suppressed before pagination.

Results expose used and missing ingredients, score breakdowns, constraint checks, warnings and proposed relaxations when no candidates remain. Hard filtering means recognised constraints are not traded against similarity, but it does not guarantee that recognition, cached metadata or underlying data is correct. The live dietary mismatch documented later demonstrates this distinction.

6.6 Conversational Assistant and Recommendations

6.6.1 Explicit conversation state

The assistant retains a parsed query, shown recipe IDs, a focused recipe ID and a turn count. The frontend sends state back with the next message and persists the conversation in session storage. The chat API translates this state into the internal representation. Context is therefore explicit application data, not hidden LLM memory.

Each turn checks reset phrases, references to shown results, recipe questions and filter removal before parsing and merging new constraints. Ordinals such as first, second and last resolve against the displayed IDs. Answers about cooking time, ingredients, nutrition and instructions are constructed from stored recipe fields and derived metadata. Unknown nutrition or servings are stated as unknown.

6.6.2 Follow-up merge rules

The newest role wins for a repeated ingredient. An "only" refinement makes named ingredients required. Newly specified cuisine, meal type or time replaces the corresponding field; nutrition constraints replace earlier constraints for the same nutrient. Diets, exclusions and qualities are combined with special handling for incompatible changes. A new-topic heuristic preserves earlier exclusions and diets while replacing other context.

A live check started with "I have rice, tomatoes and onions" and then added "Under 20 minutes". The displayed state retained the three available ingredients and added a 20-minute maximum. The match count changed from 5,594 to 1,534 in that session. This is a functional observation, not an accuracy benchmark or latency result. The chatbot search and answer path did not invoke an LLM.

6.6.3 Content-based recommendations

The recommendation endpoint forms a normalised mean embedding of favourited or highly rated recipes, then scores candidate recipe vectors against it. Liked cuisines add 0.3 and liked ingredients add 0.2. The ranker applies saved dietary and allergy constraints first and excludes previously interacted-with recipes. If there is no positive preference signal, the endpoint returns an empty recommendation list.

Recipe similarity combines 0.6 semantic similarity with 0.4 Jaccard overlap of essential ingredients. Comparison returns shared and unique ingredients, estimated attributes and optional semantic similarity. These are content-based mechanisms; no collaborative-filtering training, user-embedding training or recommendation-specific human study was found.

6.7 Recipe Modification and Generation

6.7.1 Rule-based adaptation

RecipeModifier interprets substitution, removal, diet, spice, scaling, quicker and lighter requests. It uses a curated substitutions.json knowledge base, updates ingredient lines, rewrites matching instruction text and produces a before/after change list. Diet validation is rerun on the result. Unsupported transformations and unresolved substitutions are reported rather than silently treated as successful.

Serving scaling changes parsed leading quantities when an original serving count is known; otherwise the system asks for clarification or warns that quantities were not scaled. This matters because the source corpus has no serving counts. A textual substitution is not proof of culinary equivalence, recipe quality or safe preparation.

6.7.2 Actual generation defaults

The current Generate page initialises **useLlm** to true, and the API request schema also defaults **use_llm** to true. The local LLM is used when requested and available. Deselecting it, lacking a local model/runtime, or encountering a runtime failure leads to a labelled retrieve-and-adapt path. The existing README calls adaptation the default; the current code is more precise and takes precedence in this report.

6.7.3 Optional local generative path

The runtime searches for an explicit GGUF model path or chooses from locally present files, preferring Qwen3-4B-Instruct-2507-Q4_K_M.gguf, then Llama-3.2-3B-Instruct-Q4_K_M.gguf, then Qwen3-1.7B-Q4_K_M.gguf. A llama.cpp subprocess binds to localhost. Generation uses retrieved examples, a JSON schema, Pydantic validation, ingredient/constraint checks and at most one repair round.

Grounding retrieves up to three recipes after making available/required ingredients soft preferences and dropping the time limit; cuisine is relaxed if no examples remain. Output validation, rather than the grounding examples alone, checks requested use and timing constraints. A single-worker queue runs jobs in memory, and the frontend polls job status. These jobs do not constitute a durable background-job system.

6.7.4 Evidence boundary

The generation benchmark script and 16 labelled requests exist, but completed quality-result files were not found. No generation adherence, taste, safety or model-quality score is claimed. No generative request, expensive inference or model download was performed for this report. Retrieve-and-adapt can still fail requested constraints and exposes validation status accordingly.

6.8 Meal Planning and Shopping List

The planner parses one to seven days, defaults to three days, and uses requested breakfast/lunch/dinner slots or dinner alone. For each slot it obtains up to 40 hybrid-ranked candidates. It greedily avoids previously selected IDs, limits a cuisine to max(2, days // 2), limits a main protein ingredient to two uses and generally avoids repeating the last title word except salad, rice or bowl.

If variety rules cannot be satisfied, it chooses another unused matching recipe and explains the repeated cuisine/protein. If no matching recipe is available, the slot remains empty. This is a greedy scheduling heuristic, not a global optimisation method or a nutritionally balanced meal-plan solver.

The shopping list counts canonical essential ingredients needed across selected recipes, excludes recognised ingredients available in the request and groups items by lexicon category. Counts represent the number of recipes needing an item. Quantity totals, serving-adjusted purchases and nutrition totals are deliberately not calculated.

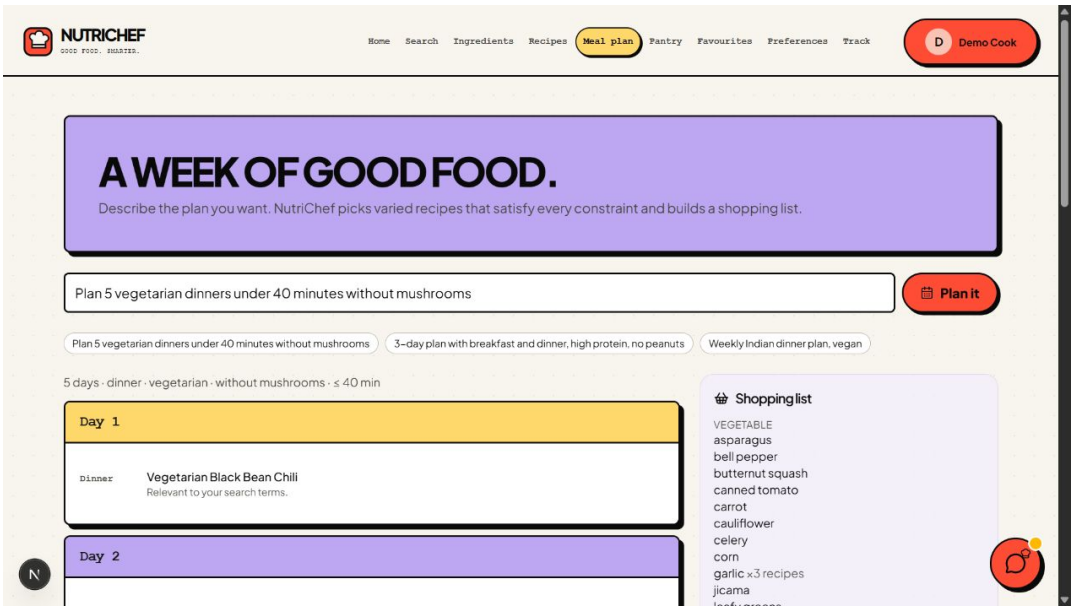


Figure 3. Actual local meal-planner screen for five vegetarian dinners under 40 minutes without mushrooms. The viewport shows the first day slots and shopping-list panel.

6.8.1 Observed planning limitation

The live five-day request produced a Day 5 entry titled "Grilled Fish Tostadas with Pineapple-Jicama Salsa", and its shopping list included cod. This contradicts the vegetarian request despite the planner reusing ranker constraints. It is a verified application-level inconsistency requiring investigation of recipe-feature alignment or filtering; its root cause was not established or repaired during report preparation. The feature is implemented, but dietary correctness must not be presented as guaranteed.

7. Implementation

Technology	Purpose verified in the repository
TypeScript, React 19, Next.js 15.5.2	App Router interface, application APIs and same-origin proxy
Tailwind CSS 4, Lucide, Framer Motion	Shared responsive styling, icons and motion with reduced-motion support
Python, FastAPI, Pydantic	NLP service, endpoint contracts and structured validation
scikit-learn, NumPy, SciPy	TF-IDF, vector scoring and sparse matrix updates
Sentence Transformers / PyTorch	Local pretrained MiniLM inference
RapidFuzz	Damerau-Levenshtein domain spelling correction
PostgreSQL, Prisma, pg; SQLite	Application records; recipe corpus and NLP user data
pytest, JSONL evaluation scripts	Unit/API tests and saved reproducible evaluation artefacts

The application runs through local services at localhost:3000, 127.0.0.1:8000 and local PostgreSQL on port 5433. Source recipes, user recipes, version history, favourites, preferences, pantry and tracking use the distinct storage paths described in Figure 1. Generation remains available at /generate and the assistant is a floating panel rather than a main-navigation entry.

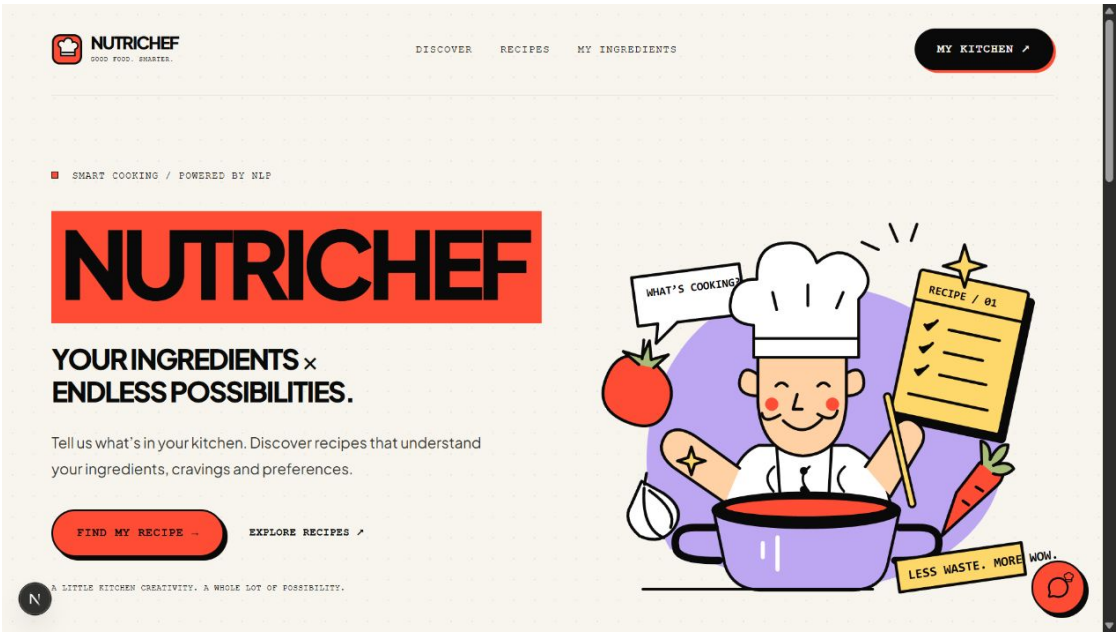


Figure 4. Genuine running homepage after the frontend redesign. The food illustrations are local SVG assets, not photographs from the source dataset.

7.1 Application Screenshots and Evidence

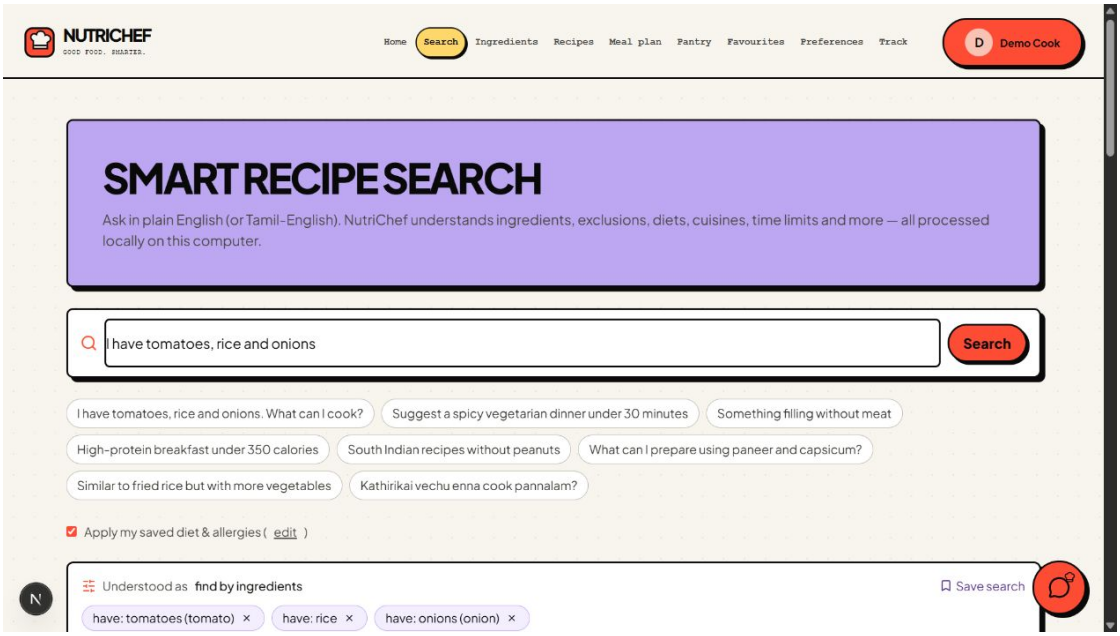


Figure 5. Live natural-language search with "I have tomatoes, rice and onions". The interpretation chips expose tomato/onion canonicalisation and availability roles. Results were observed in the same session; this capture is the search viewport.

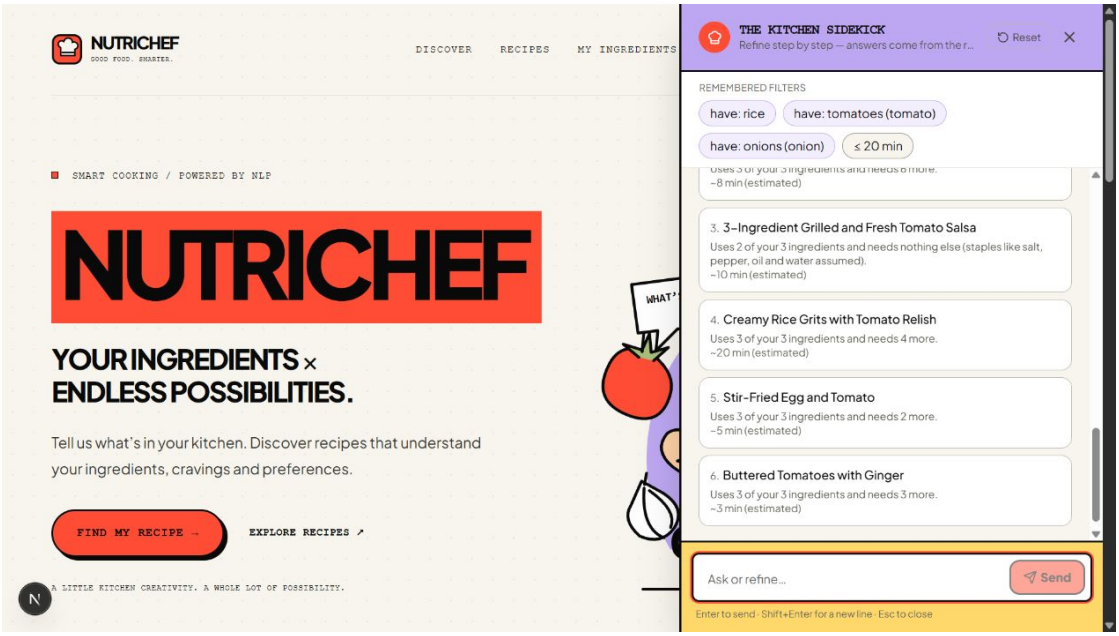


Figure 6. Genuine floating conversational panel after adding "Under 20 minutes". Remembered ingredient chips and the time constraint are retained while recipe suggestions update.

Screenshots were captured from the existing local demo account. They are implementation evidence, not mockups. Live search returned Tomato and Parmesan Risotto and other stored recipes; the assistant returned grounded suggestions and maintained the follow-up time constraint. These checks cover the demonstrated interactions only. Recipe editing, recommendation quality and all account workflows were not comprehensively retested.

8. Experimental Results

Results below are transcribed from **eval_dev.json** and **eval_test_heldout.json** and checked against the corresponding Markdown files and evaluator. Both runs index 13,475 recipes. Recorded timestamps are 22 October 2026 at 21:45 and 21:46 respectively; the saved files do not specify a timezone. No full benchmark rerun was performed for this report.

Task	Development set	Held-out test set
Ingredient extraction	45 requests	35 requests
Query understanding	35 requests	30 requests
Synonym normalisation	30 cases	34 cases
Spelling correction	20 errors + 10 protected words	20 errors + 10 protected words
Retrieval	15 queries, top 10	15 queries, top 10
Generation data only	8 requests; no completed results	8 requests; no completed results

8.1 Held-out ingredient extraction

System / scoring scope	Precision	Recall	F1
NutriChef: name and role	0.894	0.916	0.905
NutriChef: name only	0.929	0.952	0.940
Original normaliser: name only	0.435	0.446	0.440
spaCy NER baseline: name only	0.105	0.024	0.039

Name-and-role counts are TP = 76, FP = 9 and FN = 7, reproducing the reported micro precision, recall and F1. Quantity-plus-unit accuracy is 1.000 over 12 annotated quantities. The general-purpose spaCy baseline is not a trained ingredient recogniser; its low score supports domain modelling but does not compare against a specialised food NER model.

8.2 Query understanding and normalisation

Held-out intent accuracy is 0.967 (29/30). Recorded field accuracy is 1.000 for diet (n=9), meal type (n=14), cuisine (n=4), time limit (n=3) and qualities (n=6); nutrition is 0.667 (n=3). Tanglish detection is 1.000 on only two annotated examples. These denominators are field-specific, not the full query-set size.

Synonym accuracy is 0.912 (31/34), including Tamil aliases 0.889 (8/9) and British aliases 0.778 (7/9). Typo correction is 1.000 on 20 misspellings, and 10 correctly spelled words remain unchanged. These small targeted sets are insufficient to establish universal normalisation accuracy.

8.3 Retrieval Results and Metric Definitions

Method	P@5	P@10	MRR	nDCG@10*	Violation
Keyword	0.413	0.373	0.533	0.400	0.500
TF-IDF	0.547	0.440	0.629	0.478	0.517
MiniLM	0.480	0.407	0.565	0.426	0.667
Hybrid	0.827	0.800	0.917	0.819	0.167

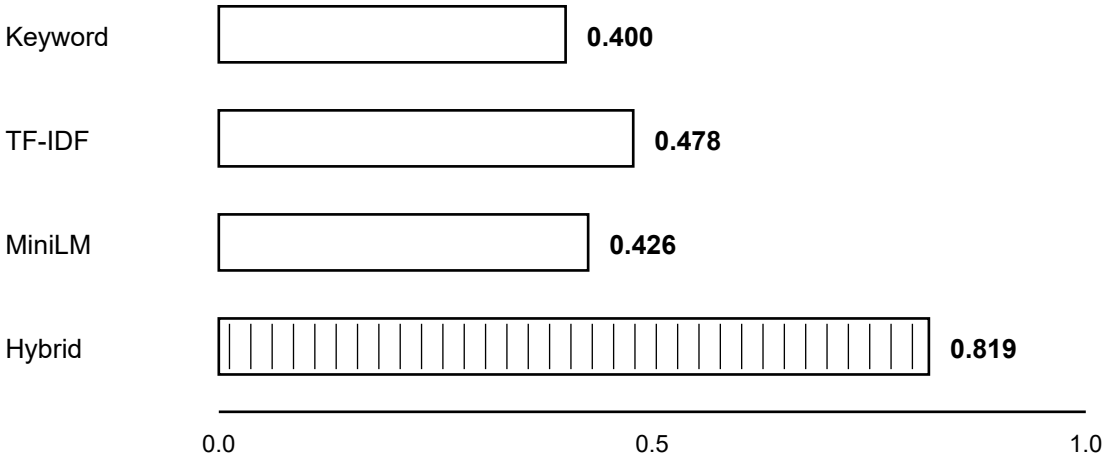


*Figure 7. Saved held-out reported nDCG@10 values (15 queries). *The evaluator normalises by an ideal list of ten relevant results; see the definition below.*

Method	p50 latency (ms)	p95 latency (ms)	Dev nDCG@10*
Keyword	7.9	12.5	0.254
TF-IDF	10.9	12.5	0.424
MiniLM	10.3	11.9	0.314
Hybrid	32.4	40.1	0.817

Relevance: the evaluator matches independently specified required term groups and excluded words against raw title and ingredient text. P@5 and P@10 pad missing results as non-relevant. MRR uses the first relevant item within the evaluated list. DCG discounts binary relevance by log2(rank + 1). The reported nDCG divides by an all-relevant ten-result list, even if fewer than ten relevant recipes exist; it is not necessarily conventional query-specific nDCG.

Violations: the rate is averaged only across queries with exclusions (6/15 held-out; 7/15 development), using the fraction of returned items containing an excluded raw-text term. Latency is measured around ranker.search, not end-to-end HTTP or page rendering. The p95 implementation uses a sorted-list index on only 15 samples, so tail-latency interpretation is limited.

9. Performance Evaluation and Discussion

On the saved held-out set, hybrid reported nDCG@10 exceeds TF-IDF by 0.341 and MiniLM by 0.393. It also reduces the recorded exclusion violation rate to 0.167, while increasing median ranker time to 32.4 ms. These results support the benefit of the complete constraint-aware pipeline on these queries. They do not isolate the contribution of score fusion: baselines use normalised query text without the hybrid constraint masks or ingredient scoring.

Evaluation labels were AI-authored and are not independently human-verified. The retrieval sets contain only 15 queries per split and measure raw-text constraint matching rather than taste, recipe usefulness or user satisfaction. Lexical checks may produce false alarms such as matching butter inside peanut butter. Conversely, lexicon gaps, ambiguous dishes and metadata errors can produce genuine misses. No confidence intervals, statistical-significance test or human recommendation study was found.

9.1 Historical results versus current code

Saved failures include chick peas, the direction of "no more than 250 calories", coriander negation, juice-of-lemon parsing and an unrecognised generate intent. Current code contains chick peas, swede and mangetout aliases and explicit handling of "no more than". The original README identifies several of these as post-evaluation fixes. The saved values must therefore remain historical; they are not revised upward without a fresh held-out run.

The live meal-plan check included a fish recipe and cod under a vegetarian request. This concrete counterexample prevents an unconditional claim that the present application enforces dietary correctness. Local inference also has startup and resource costs beyond query-time latency. The saved warm-cache engine build was 23.3 seconds on the held-out run, not the latency of an individual search.

10. Conclusion

NutriChef implements an integrated local NLP recipe platform with domain normalisation, structured intent and constraints, lexical and dense retrieval, explainable ranking and conversational refinement. The saved evaluation favours its complete hybrid pipeline on the small project-specific test set. The strongest defensible outcome is a functioning, inspectable prototype with measured strengths and documented data, evaluation and filtering limitations.

11. Future Scope

Priorities are to investigate the observed planning/filter mismatch; expand human-validated, adversarial multilingual test sets; add phrase-aware allergen evaluation; improve recipe-feature consistency checks; evaluate ranking ablations; and complete controlled generation-quality studies. Nutrition-aware planning requires trustworthy nutrition and serving data. Larger-corpus performance optimisation and broader Tamil understanding remain future work.

12. References and Repository Evidence

[1] C. D. Manning, P. Raghavan and H. Schutze, *Introduction to Information Retrieval*. Cambridge University Press, 2008. Online: https://nlp.stanford.edu/IR-book/.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *EMNLP-IJCNLP*, 2019, pp. 3982-3992. doi: 10.18653/v1/D19-1410. https://aclanthology.org/D19-1410/.

[3] Sentence Transformers, "all-MiniLM-L6-v2," official model card. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2. Accessed 22 Oct. 2026.

[4] RapidFuzz, "Damerau Levenshtein," official API documentation. https://rapidfuzz.github.io/RapidFuzz/Usage/distance/DamerauLevenshtein.html. Accessed 22 Oct. 2026.

[5] S. Bruch, S. Gai and A. Ingber, "An Analysis of Fusion Functions for Hybrid Retrieval," arXiv:2210.11934, 2022. https://arxiv.org/abs/2210.11934.

[6] D. Jannach, A. Manzoor, W. Cai and L. Chen, "A Survey on Conversational Recommender Systems," *ACM Computing Surveys*, vol. 54, no. 5, art. 105, 2021. doi: 10.1145/3453154. https://arxiv.org/abs/2004.00646.

[7] Kaggle, "Food Ingredients and Recipes Dataset with Images," dataset listing. https://www.kaggle.com/datasets/pes12017000148/food-ingredients-and-recipe-dataset-with-images. Dataset attribution is recorded in the project README; local counts were independently checked.

[8] scikit-learn developers, "TfidfVectorizer" and "Feature extraction," official documentation. https://scikit-learn.org/stable/modules/feature_extraction.html. Accessed 22 Oct. 2026.

[9] S. Divyabanu, *NutriChef*, project repository. https://github.com/sDivyabanu/NutriChef. Implementation statements in this report are based on the inspected local working tree, not an assertion that all local changes are published.

Repository evidence inspected

Dataset and import: data/13k-recipes.csv; backend/scripts/build_corpus.py; backend/data/corpus_report.json. Architecture: next.config.ts; prisma/schema.prisma; backend/app/db.py; main.py; services.py. NLP: backend/app/nlp/{text, ingredients, lexicon, spelling, query, retrieval, ranker, engine, chat, modify, generate, mealplan, similarity}.py and the ingredient/substitution resources.

Evaluation: backend/eval/run_eval.py; eval/data/*.jsonl; eval/results/eval_dev.{json,md}; eval_test_heldout.{json,md}; bench_llm.py. Tests: backend/tests and scripts/smoke-crud.mjs. The test sources contain 18 test functions, expanding to 21 cases with parametrisation. A fresh unit-test attempt could not start because the stored virtual-environment interpreter path no longer resolved; historical README pass claims were not treated as new test evidence.